

AI ASSISTED CODING

SUMANTH POLAM

2303A51121

BATCH – 03

10 – 03 – 2026

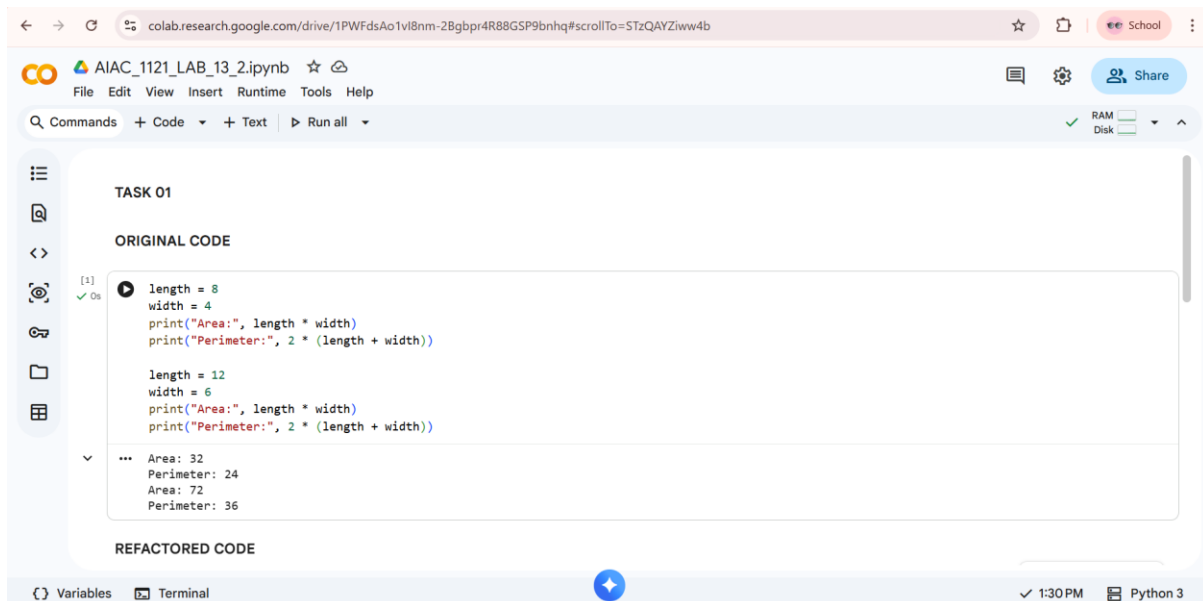
ASSIGNMENT – 13.2

LAB – 13 : Code Refactoring – Improving Legacy Code with AI Suggestions.

Task – 01 : Refactoring – Eliminating Repetitive Logic.

Prompt : Use AI to analyze the Python code and identify repeated calculations for area and perimeter. Refactor the code by creating a reusable function with proper docstrings while ensuring the output remains the same.

Original Code & Output :



The screenshot shows a Google Colab notebook titled "AIAC_1121_LAB_13_2.ipynb". The code is labeled "TASK 01" and "ORIGINAL CODE". It contains two sets of calculations for area and perimeter of rectangles. The first set uses length=8 and width=4, and the second set uses length=12 and width=6. The output shows the area and perimeter for each set.

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Output:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Refactored Code & Output :



The screenshot shows the same Google Colab notebook, but the code is now labeled "REFACTORED CODE". It uses a reusable function named "calculate_rectangle" to calculate the area and perimeter of rectangles. The function takes length and width as arguments and returns the area and perimeter. The output is the same as the original code.

```
def calculate_rectangle(length, width):
    area = length * width
    perimeter = 2 * (length + width)

    print("Area:", area)
    print("Perimeter:", perimeter)

calculate_rectangle(8, 4)
calculate_rectangle(12, 6)
```

Output:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

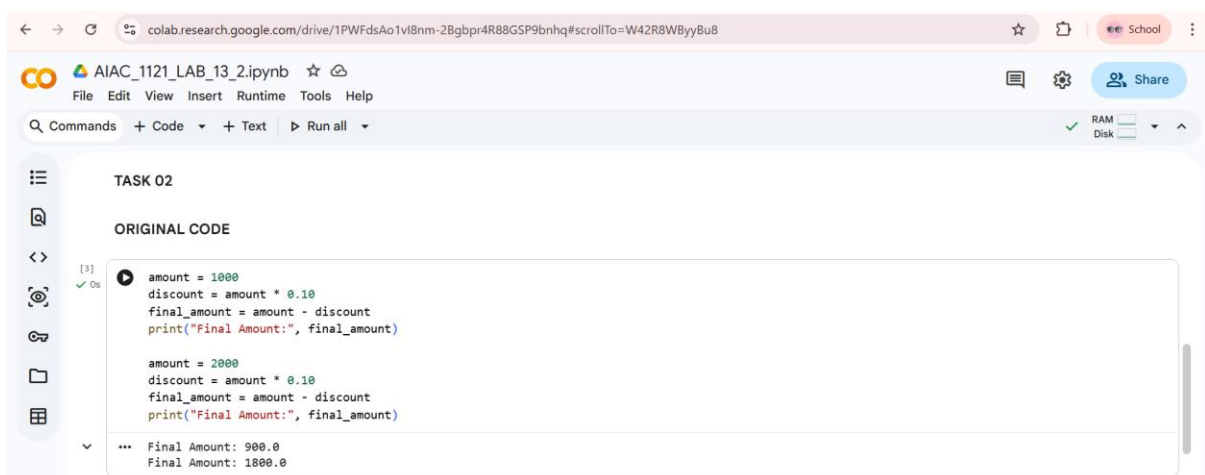
Explanation :

The repeated calculations for area and perimeter were moved into a reusable function called `calculate_rectangle`. This improves code readability and reduces duplication. The program still produces the same output but is easier to maintain.

Task – 02 : Refactoring – Modularizing Inline Calculations.

Prompt : Use AI to refactor the Python code by identifying repeated discount calculations and converting them into a reusable function with proper documentation.

Original Code & Output :

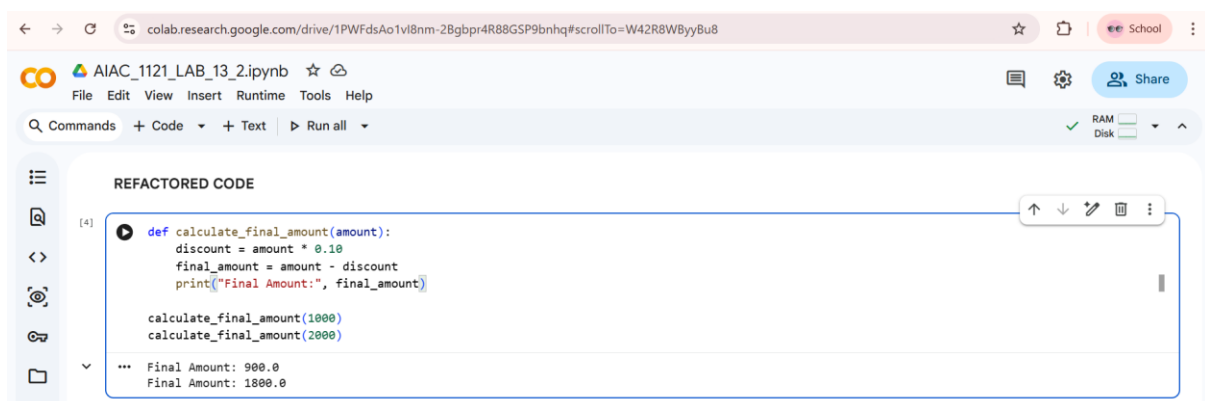


```
colab.research.google.com/drive/1PWFDsAo1vI8nm-2Bgbpr4R88GSP9bnhq#scrollTo=W42R8WByyBu8
AIAC_1121_LAB_13_2.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
RAM Disk
TASK 02
ORIGINAL CODE
[3] 0s
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

... Final Amount: 900.0
Final Amount: 1800.0
```

Refactored Code & Output :



```
colab.research.google.com/drive/1PWFDsAo1vI8nm-2Bgbpr4R88GSP9bnhq#scrollTo=W42R8WByyBu8
AIAC_1121_LAB_13_2.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
RAM Disk
REFACTORED CODE
[4] 0s
def calculate_final_amount(amount):
    discount = amount * 0.10
    final_amount = amount - discount
    print("Final Amount:", final_amount)

    calculate_final_amount(1000)
    calculate_final_amount(2000)

... Final Amount: 900.0
Final Amount: 1800.0
```

Explanation :

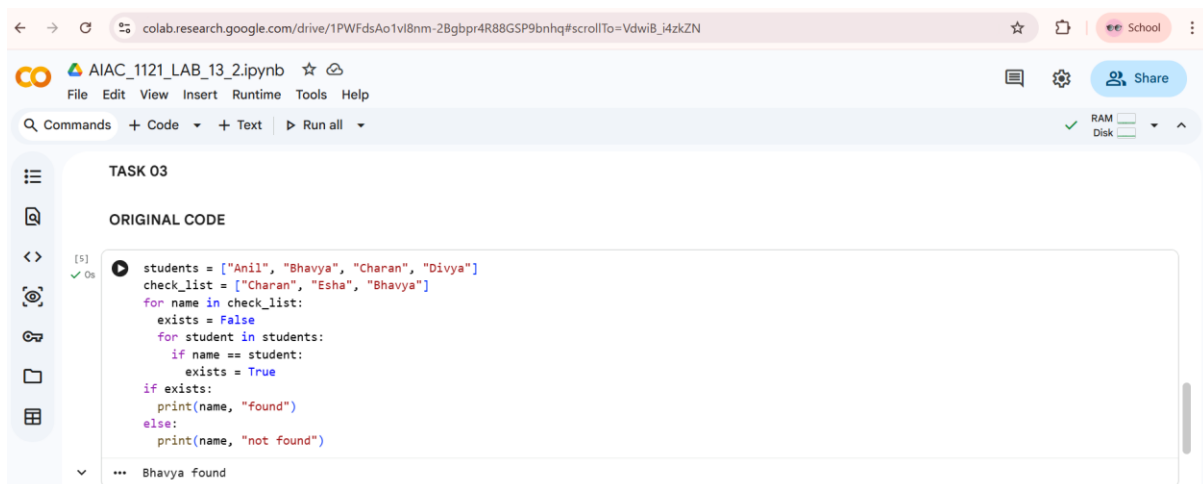
The repeated discount calculation logic was moved into a function called `calculate_final_amount`. This makes the code modular and reusable for

different input values. It also improves readability and reduces redundant code.

Task – 03 : Refactoring – Improving Loop and Conditional Performance.

Prompt : Use AI to optimize the Python program by replacing inefficient nested loops with a faster lookup structure while keeping the output unchanged.

Original Code & Output :



```
colab.research.google.com/drive/1PWFDsAo1vI8nm-2Bgbpr4R88GSP9bnhq#scrollTo=VdwiB_i4zkZN
```

AIAC_1121_LAB_13_2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

TASK 03

ORIGINAL CODE

```
[5] ✓ Os
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

... Bhavya found

Refactored Code & Output :



```
colab.research.google.com/drive/1PWFDsAo1vI8nm-2Bgbpr4R88GSP9bnhq#scrollTo=VdwiB_i4zkZN
```

AIAC_1121_LAB_13_2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

REFACTORED CODE

```
[6] ✓ Os
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

student_set = set(students)

for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
```

... Charan found
Esha not found
Bhavya found

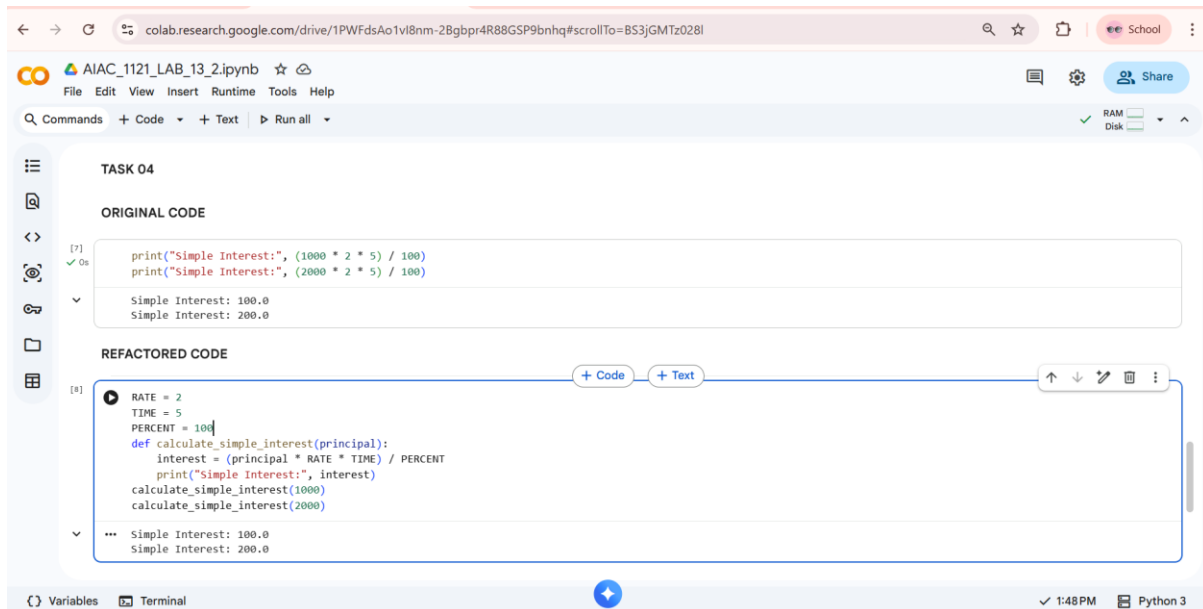
Explanation :

The nested loops were replaced with a set data structure, which provides faster lookup operations. Instead of checking each student repeatedly, membership is checked directly using `in`. This improves performance from $O(n^2)$ to $O(n)$.

Task – 04 : Refactoring – Replacing Hardcoded Values with Constants.

Prompt : Use AI to identify hardcoded numerical values in the simple interest calculation and replace them with descriptive constants to improve maintainability.

Original/Refactored Code & Output :



The screenshot shows a Google Colab notebook titled "AIAC_1121_LAB_13_2.ipynb". It displays two code cells under the heading "TASK 04".

ORIGINAL CODE

```
[7]: print("Simple Interest:", (1000 * 2 * 5) / 100)
     print("Simple Interest:", (2000 * 2 * 5) / 100)
```

The output for the original code is:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

REFACTORED CODE

```
[8]: RATE = 2
     TIME = 5
     PERCENT = 100
     def calculate_simple_interest(principal):
         interest = (principal * RATE * TIME) / PERCENT
         print("Simple Interest:", interest)
     calculate_simple_interest(1000)
     calculate_simple_interest(2000)
```

The output for the refactored code is:

```
Simple Interest: 100.0
Simple Interest: 200.0
```

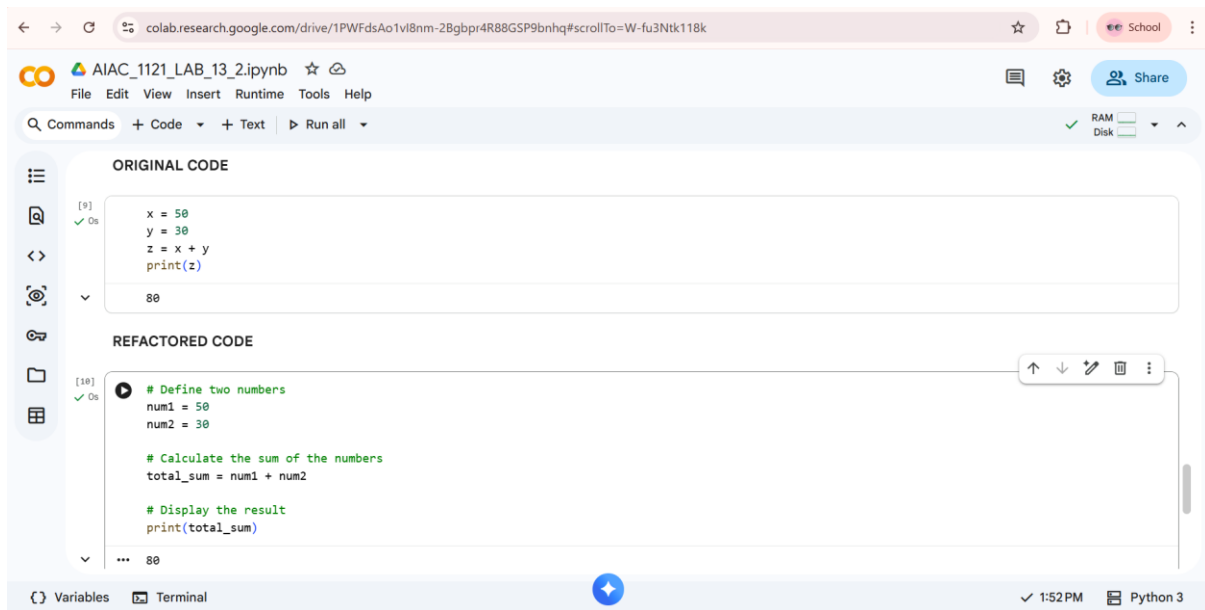
Explanation :

Hardcoded values such as rate, time, and percentage were replaced with named constants (RATE, TIME, PERCENT). This makes the code easier to understand and modify. If the rate or time changes, it only needs to be updated once.

Task – 05 : Refactoring – Enhancing Variable Naming Code Clarity.

Prompt : Use AI to improve code readability by replacing unclear variable names with meaningful ones and adding comments explaining the logic.

Original/Refactored Code & Output :



The screenshot displays a Google Colab notebook interface. The top bar shows the file name 'AIAC_1121_LAB_13_2.ipynb' and standard menu options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu, there's a toolbar with icons for running code, saving, and sharing. The notebook is divided into two main sections: 'ORIGINAL CODE' and 'REFACTORED CODE'. The 'ORIGINAL CODE' section contains a simple Python script that defines variables x, y, and z, calculates z = x + y, and prints z. The 'REFACTORED CODE' section shows the same logic but with more descriptive variable names (num1, num2, total_sum) and added comments explaining each step. The output of both code blocks is the number 80. The bottom status bar indicates the current environment is Python 3 and the time is 1:52 PM.

```
[9] ✓ Os
x = 50
y = 30
z = x + y
print(z)

80
```

```
[10] ✓ Os
# Define two numbers
num1 = 50
num2 = 30

# Calculate the sum of the numbers
total_sum = num1 + num2

# Display the result
print(total_sum)

... 80
```

Explanation :

The variables x, y, and z were renamed to num1, num2, and total_sum to improve clarity. Comments were added to explain each step of the code. This makes the program easier to understand for other developers.

THANK YOU!!